PRUNEAGENT: AGENTIC LAYER-WISE PRUNING AND QUANTIZATION FOR LLMS

Steph Xu

1.125 Agentic Engineering & Architecture

Term Project Proposal (PS06)

MIT SLOAN SCHOOL OF MANAGEMENT

MIT MANAGEMENT SLOAN SCHOOL

THE CONSTRAINT OF ON-DEVICE LLM INFERENCE IS MEMORY-BOUND

The Target Users

Developers deploying open LLMs on a single GPU or laptop.

Students, researchers and small teams without GPU clusters.

Memory, not accuracy, is the binding constraint.

The Bottleneck

Default compression is uniform across layers.

Wanda[1] pruning and quantization[6] apply one sparsity ratio or bit width to every layer.

Layer sensitivity varies[2], so uniform settings lose more quality than necessary.

The Gap

Adaptive allocation is rule-based, compute-heavy or closed.

OWL[2]: fixed outlier-based heuristic.

RL search (AMC[3], FastForward[4]): GPU-hours of policy training.

LLM agent (Kodathala & Vunnam[5]): no public code; weak baselines.

[1] Sun et al., 2024. [2] Yin et al., 2024. [3] He et al., 2018. [4] Yuan et al., 2025. [5] Kodathala & Vunnam, 2026. [6] Frantar et al., 2023. Full references on the last slide.

MIT MANAGEMENT SLOAN SCHOOL

THE SOLUTION: CONSTRAINT-AWARE AGENTIC MODEL COMPRESSION

Inputs, Constraints and Outputs

Input: open LLM (e.g., Qwen3-1.7B) and target compression ratio (e.g., 4× smaller).

Constraints: maximum perplexity increase per step (default 15%); cost cap ($5 per run).

Output: compressed model, dynamically pruned (per-layer, per-kernel, per channel, etc.) sparsity and bit width, and benchmark report vs. 4 baselines at equal size.

Target not reached: returns the last accepted plan with the stop reason.

Functional Requirements

1. Profile layer sensitivity once on a calibration set.

2. LLM agent proposes the next action: prune, quantize or stop.

3. Rule-based validator checks every proposal.

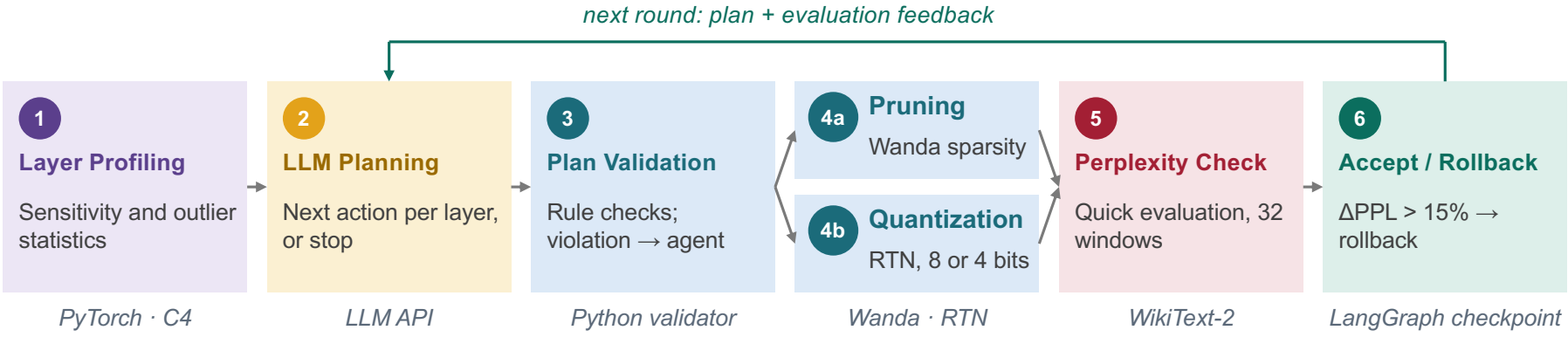
4. Apply, evaluate, and roll back any step over the perplexity limit.

5. Checkpoint every step to Google Drive or local drive; resume after disconnect.

Source: author's system architecture (ARCHITECTURE.md, ADR-001–005); Wanda and OWL reference implementations on GitHub.

TECHNICAL ARCHITECTURE

LangGraph Agent Graph one action per round: prune, quantize or stop



Final Evaluation	**lm-eval harness:** 5 zero-shot tasks + MMLU vs. uniform Wanda, OWL, magnitude pruning and uniform 4-bit RTN at equal size

Infrastructure	**Mac · development**	**Colab Pro · T4 / A100**	**Drive or local · checkpoints**	**Budget · $100 LLM, 200 CU**

MIT MANAGEMENT SLOAN SCHOOL

MAIN RISK: THE AGENT DOESN'T BEAT THE BENCHMARKS

In scope: open LLMs ≤2B parameters · unstructured pruning + 8/4-bit quantization · post-training only
Out of scope: inference speed-up · models >4B parameters · sub-4-bit quantization

Risk	Impact	Mitigation
Agent doesn't beat benchmarks	No value over uniform baselines	Midpoint check; if behind, initialize agent from OWL allocation
Colab compute units exhausted	No GPU for final runs	T4 for development; A100 for final runs only; 200-CU cap
No bf16 support on T4	Numerical errors in some layers	Final runs on A100 (bf16); fp32 fallback on T4
Perplexity misses knowledge loss	Low PPL, degraded recall	MMLU in final evaluation
Quantization branch scope creep	Week 2 overrun	RTN only, no new library; first to cut
Run time or cost overrun	>2 h or >$5 per run	Fall back to pruning only, then Qwen3-0.6B

PROJECT PLAN AND SUCCESS CRITERIA

Week 1: Baselines		Week 2: Agent Graph		Week 3: Tuning	Week 4: Evaluation
1 **Environment Setup**	2 **Baseline Implementation**	3 **Agent Graph Build**	4 **Accept / Rollback Loop**	5 **Prompt Tuning**	6 **Final Evaluation & Demo**
Mac + Colab Pro; load models; adapt Wanda code	Uniform Wanda, OWL, magnitude, 4-bit RTN; eval harness	LLM agent, validator, prune and quantize branches	Midpoint: end-to-end run on Qwen3-0.6B	Tune agent instructions on 0.6B; freeze	Qwen3-1.7B, SmolLM2-1.7B vs. 4 baselines

Success Criteria

1. Lower perplexity and higher mean benchmark score than uniform Wanda and uniform 4-bit, both final models
2. ≤2 h on a T4 and ≤$5 LLM cost per run
3. Interrupted runs resume from the last checkpoint *Stretch: match or beat OWL*
Fallback (time or cost overrun): pruning only, same criteria; then Qwen3-0.6B

Descoping order: quantization branch, SmolLM2, then magnitude baseline. Source: author's system architecture.

APPENDIX: SPEAKER NOTES

- **2 · Problem and Audience.** The users here are developers running open LLMs on a single GPU or a laptop, where memory, not accuracy, is the binding constraint. The two standard post-training tools are pruning and quantization, and the fast defaults, Wanda and round-to-nearest, apply one setting to every layer, even though layer sensitivity varies. Adaptive allocation exists: OWL uses a fixed outlier heuristic, RL search like AMC and FastForward costs GPU-hours of policy training, and the one LLM-agent paper this year released no code and compared against weak baselines. That is the gap I'm targeting.
- **3 · Proposed Solution.** The interface is simple. Input: an open model and a target compression ratio. Constraints: the maximum perplexity increase per step and a cost cap of five dollars per run. Output: the compressed model, the per-layer sparsity and bit width, and a benchmark report against four baselines at equal size. If the target is not reachable, the system returns the last accepted plan and the stop reason. Functionally, it profiles each layer once, lets the agent propose the next action, validates it, applies it, evaluates it, and rolls back any step over the perplexity limit, with a checkpoint after every step.
- **4 · Technical Architecture.** Here is the technical architecture. It is a LangGraph agent graph, not a linear pipeline. Each round the agent picks one action: prune, quantize or stop. The validator sends rule violations back to the agent. Pruning reuses the Wanda reference code; quantization is round-to-nearest at 8 or 4 bits. Every round rebuilds the model from the dense weights, pruning first, so rollback is exact. A perplexity increase over 15 percent triggers a rollback; reaching the target, a stop action or 40 rounds triggers final evaluation on lm-eval harness against four baselines.
- **5 · Scope and Risks.** Scope is deliberately narrow: open LLMs up to 2 billion parameters, post-training only, no inference speed-up claims. The main risk is that the agent doesn't beat the benchmarks; I check at the midpoint and, if behind, initialize the agent from the OWL allocation. On compute, I have 270 Colab compute units: T4 for development, A100 for final runs only, with a 200-unit cap. If a run exceeds two hours or five dollars, I fall back to pruning only, then to Qwen3-0.6B.
- **6 · Plan and Success Criteria.** Week 1 implements the four baselines and the evaluation harness, so the measurements are trusted before the agent exists. Week 2 builds the agent graph with both branches; the midpoint milestone is an end-to-end run on Qwen3-0.6B. Week 3 tunes the agent instructions on 0.6B only, then freezes them. Week 4 runs final evaluation on Qwen3-1.7B and SmolLM2-1.7B. Success means beating uniform Wanda and uniform 4-bit at equal size on both models, within two hours and five dollars per run.

MIT MANAGEMENT SLOAN SCHOOL

REFERENCES

- **Frantar, Elias, et al.** “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.” *International Conference on Learning Representations*, 2023.
- **He, Yihui, et al.** “AMC: AutoML for Model Compression and Acceleration on Mobile Devices.” *European Conference on Computer Vision*, 2018.
- **Jafari, Sadegh, et al.** “ProfilingAgent: Profiling-Guided Agentic Reasoning for Adaptive Model Optimization.” *arXiv*:2509.05584, 2025.
- **Kodathala, Sai Varun, and Rakesh Vunnam.** “LLMs can Compress LLMs: Adaptive Pruning by Agents.” *arXiv*:2601.09694, 2026.
- **Sun, Mingjie, et al.** “A Simple and Effective Pruning Approach for Large Language Models.” *International Conference on Learning Representations*, 2024. Code: github.com/locuslab/wanda.
- **Yin, Lu, et al.** “Outlier Weighed Layerwise Sparsity (OWL): A Missing Secret Sauce for Pruning LLMs to High Sparsity.” *International Conference on Machine Learning*, 2024. Code: github.com/luuyin/OWL.
- **Yuan, Xin, et al.** “FastForward Pruning: Efficient LLM Pruning via Single-Step Reinforcement Learning.” *arXiv*:2511.18977, 2025.

MIT MANAGEMENT SLOAN SCHOOL